

Discussion 3 - Programming Language Choice in AI Development

Discussion Topic:

Please select one of the following to answer for this week's discussion forum activity.

- As the AI field develops, the tools we use to share their advantages and features ultimately become more alike. Do you think it is valuable to prefer a particular deep learning toolset (i.e., Tensorflow or PyTorch) to others, and which one(s)? Is there value in learning multiple toolsets?
- Regarding the development of AI software, do you think it is valuable to prefer a particular programming language to others for these types of applications, and which one(s)? Is there value in learning multiple languages?

My Post:

Hello Class

For students developing Machine Learning (ML) models, Python may be the best choice as a primary programming language. Python has been the programming language of choice for various scientific applications, as Python's syntax is easy to read and learn, making it a better choice than other languages for individuals not having a programming background but still needing to implement code. Additionally, Python has libraries that have been developed for specific scientific fields; libraries such as NumPy, pandas, SciPy, and scikit-learn, as well as ML and Deep Learning Python-based frameworks such as TensorFlow and PyTorch, make Python an even stronger choice for scientific applications. (Python Software Foundation, n.d.). In ML, Additionally, Python libraries simplify the manipulation and processing of data and training models. Furthermore, Graphical User Interface (GUI) based software such as Jupyter Notebook and Google Colab allows users to write and execute Python code directly within their GUI, making Python code easy to use and compile (with instant, well-formatted outputs); these tools provide an interactive and easy-to-use Python environment.

It is reasonable for AI and ML students to learn, at first, one programming language, such as Python, and become proficient in it instead of dealing with the overhead of learning several languages. However, it is not one language that fits all AI and ML requirements. Thus, it is also essential for students to become familiar with other languages. Nonetheless, the programming language a student needs to specialize in or become efficient at depends on the student's professional goals. For example, Good Old-Fashioned AI (GOFAI) systems or modern symbolic AI implementations often use languages such as Lisp, Prolog, or Haskell, as these languages support the symbolic approach better by using strong logic or/and functional programming to implement rules, relationships, and knowledge (Common-Lisp.net, n.d.; Haskell.org, 2022; SWI-Prolog, n.d.). Another language example is R, which is well suited for ML implementation requiring statistics and data analysis (R Foundation, n.d.). Moreover, JavaScript is a strong choice for web-based AI implementation. The library Deeplearning4j caters to deep learning implementation in Java and the Java Virtual Machine environment. Scala blends Java programming language style class inheritance with Haskell functional programming style; Scala is well suited for AI and ML large-scale data implementation (CSU Global, n.d.). Furthermore, AI frameworks also use various languages. For

example, TensorFlow.js is a JavaScript library for ML (TensorFlow, n.d.). PyTorch, which is an open-source framework, provides a C++ interface, in addition to its Python interface (PyTorch, n.d.). Therefore, although Python may be the most suitable language for AI students starting their education in AI and ML, it is also essential throughout their academic learning journey to become familiar with several programming languages.

Additionally, with the recent emergence of AI coding harnesses such as Codex and Claude Code, becoming familiar with how to use these AI agentic coding tools is becoming essential. These AI harnesses are already being integrated into software and AI development, and they will likely, in the near future, be indispensable for AI development. Therefore, to succeed as AI develops in the next 'era', it will not be sufficient to understand how to write code and understand how to develop AI applications; it will also be necessary to know and understand how to use and collaborate with AI systems.

-Alex

References

- Anthropic. (n.d.). *Overview: Claude Code documentation*. Anthropic. <https://docs.anthropic.com/en/docs/claude-code/overview>
- Common-Lisp.net. (n.d.). *Tutorials*. Lisp. <https://common-lisp.net/tutorials>
- Colorado State University Global. (n.d.). *Lecture 3: Tools in AI development* [Interactive lecture]. Canvas.
- Google. (n.d.). *Google Colab*. Google Research. <https://research.google.com/colaboratory/faq.html>
- Haskell.org. (2022). *ACM SIGPLAN Haskell Symposium 2022*. ACM. <https://www.haskell.org/haskell-symposium/2022/index.html>
- OpenAI. (2026, February 11). *Harness engineering: Leveraging Codex in an agent-first world*. OpenAI. <https://openai.com/index/harness-engineering/>
- Project Jupyter. (n.d.). *Project Jupyter*. Jupyter. <https://jupyter.org/>
- PyTorch. (n.d.). *PyTorch C++ API*. PyTorch. <https://docs.pytorch.org/cppdocs/>
- Python Software Foundation. (n.d.). *Applications for Python*. Python Foundation. <https://www.python.org/about/apps/>
- R Foundation. (n.d.). *R: The R Project for Statistical Computing*. R Foundation. <https://www.r-project.org/>
- SWI-Prolog. (n.d.). *Using SWI-Prolog commercially*. SWI-Prolog. <https://www.swi-prolog.org/commercial/>
- TensorFlow. (n.d.). *TensorFlow.js*. TensorFlow. <https://www.tensorflow.org/js>